

Web Scraping Exercise

Web Scraping allows you to gather large volumes of data from diverse and real-time online sources. This data can be crucial for enriching your datasets, filling in gaps, and providing current information that enhances the quality and relevance of your analysis. Web scraping enables you to collect data that might not be readily available through traditional APIs or databases, offering a competitive edge by incorporating unique and comprehensive insights. Moreover, it automates the data collection process, saving time and resources while ensuring a scalable approach to continuously updating and maintaining your datasets.

Ethical web scraping involves respecting website terms of service, avoiding overloading servers, and ensuring that the collected data is used responsibly and in compliance with privacy laws and regulations.

Use Python, `requests`, `BeautifulSoup` and/or `pandas` to scrape web data:

Import Libraries

```
In [1]: import requests
        from bs4 import BeautifulSoup
        import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
```

Define the Target URL

```
In [2]: url = "https://www.inflationtool.com/rates/germany/historical"
```

Send a Request to the Website

Do not forget to check the response status code

```
In [3]: headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
        }

        response = requests.get(url, headers=headers)

        print(f"Status Code: {response.status_code}")
        if response.status_code == 200:
            print("Connection successful.")
        else:
            print("Connection failed.")
```

```
Status Code: 200
Connection successful.
```

Parse the HTML Content

Use a library to access the HTML content

```
In [4]: soup = BeautifulSoup(response.text, 'html.parser')

table = soup.find('table')

rows = table.find_all('tr')
inflation_data = []

for row in rows[1:]:
    cols = row.find_all(['td', 'th'])
    cols = [ele.text.strip() for ele in cols]

    if len(cols) >= 2:
        jahr = cols[0]
        rate_str = cols[1]
        rate_str = rate_str.replace('%', '').replace(',', '.').strip()

        try:
            if int(jahr) >= 1993:
                inflation_data.append([int(jahr), float(rate_str)])
        except ValueError:
            continue

df_inflation = pd.DataFrame(inflation_data, columns=['Year', 'Inflation Rate (Pe
df_inflation = df_inflation.sort_values(by='Year').reset_index(drop=True)

df_inflation.head(10)
```

```
Out[4]:
```

	Year	Inflation Rate (Percent)
0	1993	4.30
1	1994	2.45
2	1995	1.51
3	1996	1.49
4	1997	2.07
5	1998	0.36
6	1999	1.19
7	2000	2.00
8	2001	1.61
9	2002	1.14

Identify the Data to be Scraped

Write a couple of sentence on the data you want to scrape

I want to scrape the inflation rate in Germany from 1993 to 2025 in order to compare it with the corresponding annual beer prices.

Extract Data

Find specific elements and extract text or attributes from elements (handle pagination if necessary)

```
In [5]: table = soup.find('table')
rows = table.find_all('tr')

inflation_data = []

for row in rows[1:]:
    cols = row.find_all(['td', 'th'])
    cols = [ele.text.strip() for ele in cols]

    if len(cols) >= 2:
        year = cols[0]
        rate_str = cols[1]

        rate_str = rate_str.replace('%', '').replace(',', '.').strip()

        try:
            if int(year) >= 1993:
                inflation_data.append([int(year), float(rate_str)])
        except ValueError:
            continue

print(f"Data successfully extracted. Found lines: {len(inflation_data)}")
```

Data successfully extracted. Found lines: 33

Store Data in a Structured Format

Give a brief overview of the data collected (e.g. count, fields, ...)

```
In [6]: df_inflation = pd.DataFrame(inflation_data, columns=['Year', 'Inflation Rate (Pe

df_inflation = df_inflation.sort_values(by='Year').reset_index(drop=True)

print("--- Overview of scraped data ---")
print(f"Number of rows (years): {df_inflation.shape[0]}")
print(f"Column names: {list(df_inflation.columns)}")
print("\nFirst 5 lines:")
print(df_inflation.head())

print("\nData Structure Overview:")
df_inflation.info()
```

```

--- Overview of scraped data ---
Number of rows (years): 33
Column names: ['Year', 'Inflation Rate (Percent)']

```

First 5 lines:

	Year	Inflation Rate (Percent)
0	1993	4.30
1	1994	2.45
2	1995	1.51
3	1996	1.49
4	1997	2.07

Data Structure Overview:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 33 entries, 0 to 32
```

```
Data columns (total 2 columns):
```

#	Column	Non-Null Count	Dtype
0	Year	33 non-null	int64
1	Inflation Rate (Percent)	33 non-null	float64

```
dtypes: float64(1), int64(1)
```

```
memory usage: 660.0 bytes
```

Save the Data

```

In [7]: df_inflation.to_csv('scraped_inflation_data.csv', index=False)
        print("Data successfully saved as 'scraped_inflation_data.csv'")

```

Data successfully saved as 'scraped_inflation_data.csv'